AdaGrad: A Learning Rate for Every Parameter

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. When AdaGrad helps . . . 3
4. The problem: a sparse feature stretches the loss . . . 3
    4.1 Gradient descent and momentum on the elongated bowl . . . 6
    4.2 Why: a sparse feature gives small gradients . . . 7
5. The idea: shrink the learning rate where the gradients are large . . . 8
6. The update rule . . . 9
7. Sparse features in real data: movie reviews . . . 10
8. The weakness: the learning rate only goes down . . . 11
9. AdaGrad in Keras . . . 13
10. Summary . . . 14
11. Sources . . . 14
12. Key terms . . . 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Learning rate; Optimizers in deep learning.
- **Leads to:** RMSProp.

1. Overview

Key point: AdaGrad gives every parameter its own learning rate: the global rate divided by the square root of the sum of that parameter's past squared gradients. Parameters with large gradients get small steps, parameters with small gradients (such as the weights of **sparse features** (G-1841)) get large ones. The price: the sum only grows, so the steps keep shrinking.

AdaGrad (G-168), short for *adaptive gradient* (Duchi et al. 2011), does not keep the learning rate fixed. AdaGrad adapts the **learning rate** (G-1068) to the situation, separately for every parameter. Every optimizer so far, from **batch gradient descent** (G-264) to momentum (G-1258; moves by a running average of past steps) and NAG (G-1315; momentum that measures the slope after its jump), uses one learning rate for all parameters; AdaGrad is the first that does not.

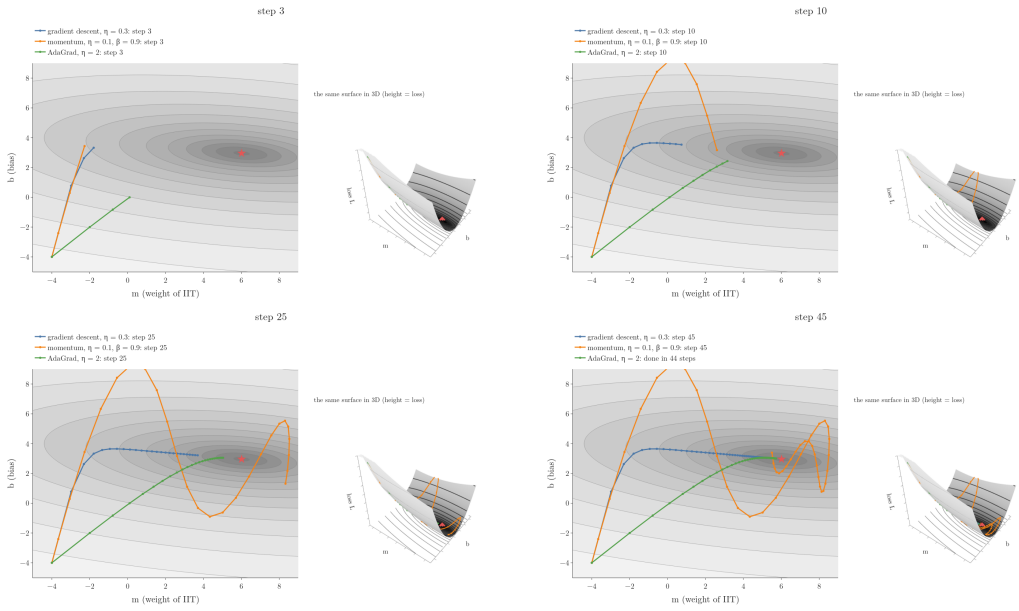


Figure 1: Gradient descent, momentum and AdaGrad on a loss stretched by a sparse feature. Gradient descent first moves along b, then crawls along m; momentum also shoots along b first, then swings in wide loops; AdaGrad heads for the minimum

Figure 1 shows where this matters: a loss shaped like a stretched bowl, produced by a feature

that is mostly zeros. Figure 1 is a **contour map** (the surface seen from above; each line joins points of equal loss; how to read a contour map), with the same surface in 3D at its right (height = the loss; the lines on it are the lines of the map); section 4 builds this surface from the data and tilts it from the side to the top view (Figure 2). Lines close together mean steep, and the star is the lowest point.

2. Prerequisites

- The weak spot one learning rate for every direction.
- Momentum's update and its **overshooting** (G-1430).
- The gradient of the squared error for a slope m and an intercept b.

3. When AdaGrad helps

> **Key point:** Two situations: features on very different scales, and sparse features, which are mostly zeros. Scaling fixes the first, so the second is the real use.

1. **Features on very different scales.** For example CGPA (0 to 10) and salary (up to lakhs of rupees) as two **features** (G-772; input variables, the columns of the data table). In practice we first standardise such features (rescale each to mean 0 and spread 1), so this case matters less.
2. **Sparse features.** A **sparse feature** (G-1841) is one whose values are mostly zero. Take students described by IQ, CGPA and whether they studied at an IIT, with the salary package as the **target** (G-1949; the output we predict). Very few students go to an IIT, so the IIT feature is 0 for almost everyone. Yet it is an important feature: an IIT student tends to get a much higher package. On data with such features, **AdaGrad** (G-168) gives better results.

AdaGrad performs larger updates for infrequent features and smaller updates for frequent ones, which is why it is well suited to **sparse data** (G-1840; Ruder 2016, §4.3).

4. The problem: a sparse feature stretches the loss

> **Key point:** With a **sparse feature** (G-1841), the loss changes slowly along that feature's weight and quickly along the others. The bowl becomes elongated, and gradient descent wastes time moving along the steep direction first.

We use a small made-up dataset, since no real dataset shows the effect this cleanly in two parameters: 100 students, each an **observation** (G-1374; one record, one row of the table), with one feature, IIT (1 for 10 students, 0 for the other 90), and the package in lakh rupees as target. The model is a single node, $\hat{y} = m \cdot \text{IIT} + b$, with the squared error as loss. Its best values are $m = 6.01$ and $b = 2.96$: about 3 lakh rupees, plus 6 for an IIT student.

The loss for any pair (m, b) is the average squared error over the $n = 100$ students, where x_i is the IIT value (0 or 1) of student i and y_i their package:

$$L(m, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (m\, x_i + b) \right)^2$$

Take $(m, b) = (-4, -4)$, the start of the paths below. The predictions for the two kinds of student:

$$\text{IIT} = 0 : \quad -4 \times 0 + (-4) = -4$$

$$\text{IIT} = 1 : \quad -4 \times 1 + (-4) = -8$$

The first student has IIT $= 0$ and package $y = 2.33$:

$$\text{error} = 2.33 - (-4) = 6.33$$

$$6.33^2 = 40.07$$

Averaging the 100 squared errors gives

$$L(-4, -4) = 72.63$$

and at the best values $L(6.01, 2.96) = 0.23$. Every pair (m, b) gives one such loss, so the loss is a surface over the (m, b) floor, with the loss as height. Figure 2 draws it for a dense feature (left) and for the sparse IIT feature (right), seen from the side and then tilted to the top view.

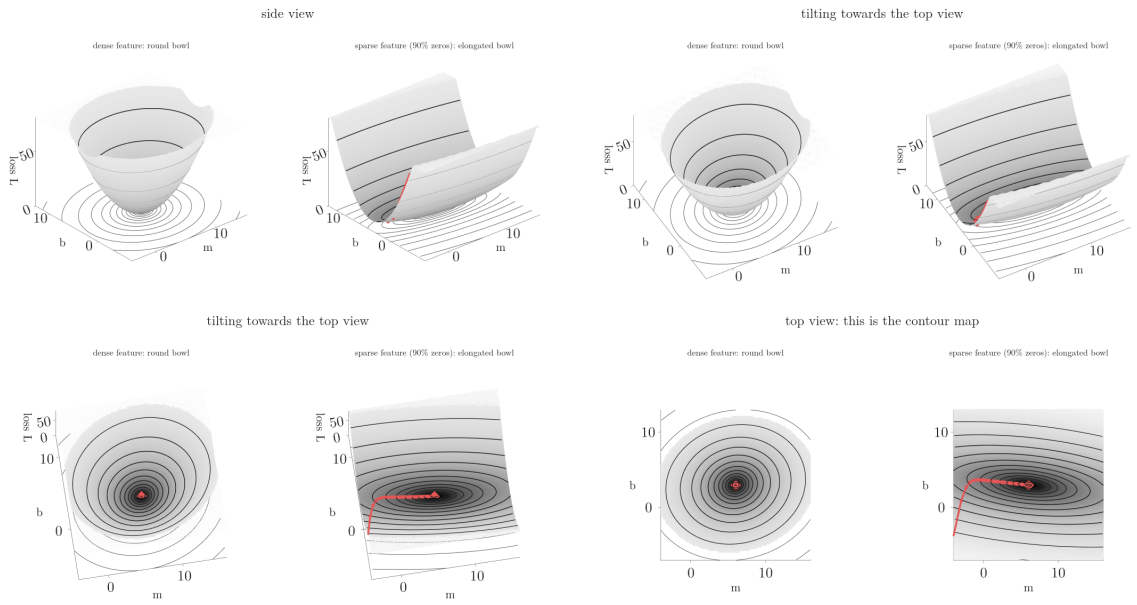


Figure 2: The two losses as surfaces, tilting from the side view to the top view. Left: a dense feature. Right: the sparse IIT feature, with gradient descent ($\eta = 0.3$) from $(-4, -4)$ in red. Height is the loss L, cut flat at 80: the walls go higher than drawn. Thin lines join points of equal height, on the surface and dropped to the floor, each about 1.6 times the one inside it; the diamond is the minimum

On the left the surface is a round bowl; on the right it is a bowl stretched along m: steep in the b direction, shallow in the m direction. The last frame is the **contour map** of each surface (how to read a contour map): the surface seen from above, where each line joins points at the same height. Lines close together mean steep; the centre ring is the lowest point. Figure 3 is these two maps on their own, now with the gradient descent path on the sparse one.

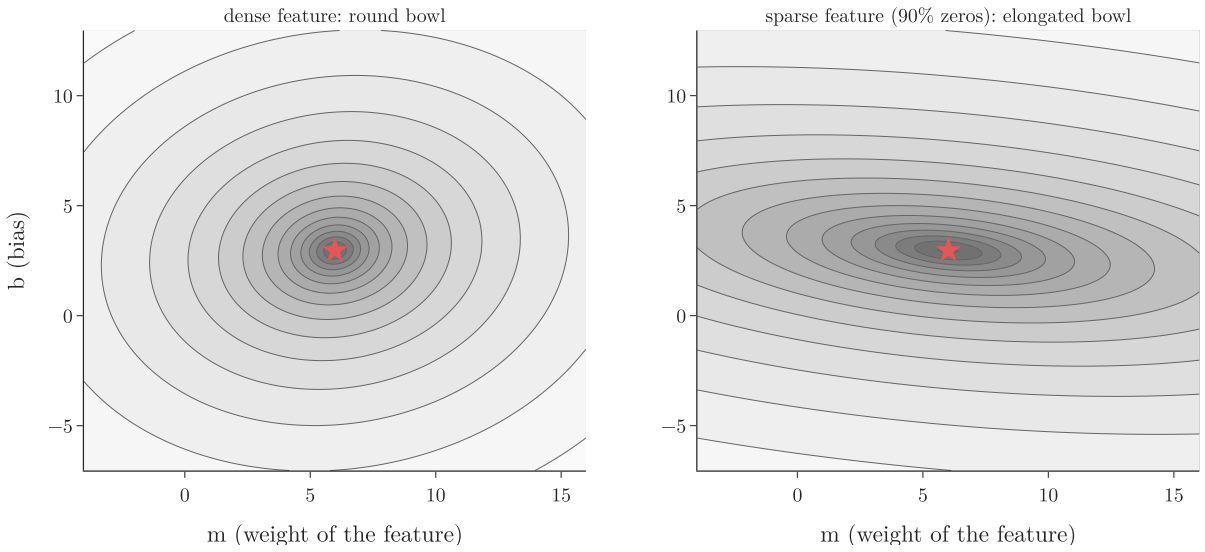


Figure 3: The loss over (m, b). Left: with a dense feature (no zeros) the contours are nearly round. Right: with the sparse IIT feature the bowl is stretched along m. Red: gradient descent ($\eta = 0.3$) from $(-4, -4)$, as drawn on the surface in Figure 2; star: the minimum

Figure 3 compares the two shapes. With a normal, dense feature the contours are nearly circles: the loss changes at a similar rate in both directions. With the sparse feature they are long ellipses: moving b changes the loss a lot, moving m changes it little. This is the **elongated bowl** (G-673).

4.1 Gradient descent and momentum on the elongated bowl

> **Key point:** Both move mostly along b first. Gradient descent then turns and crawls along m; momentum overshoots and swings in wide loops. A direct route to the minimum would be shorter.

Starting from $(m, b) = (-4, -4)$, gradient descent ($\eta = 0.3$) first moves almost only in b: after 10 steps b is at 3.54, already past its best value, while m has only reached 0.74. Then it slowly travels along m (Figure 1, blue). Momentum ($\eta = 0.1$, $\beta = 0.9$, orange) also starts almost straight up along b, but its speed carries b far past its best value, to about 9, and it then swings in wide loops around the minimum. Gradient descent's path looks like an "L", momentum's like a loop; a straight line to the minimum would be shorter than either (Notebook).

4.2 Why: a sparse feature gives small gradients

> **Key point:** The gradient for m sums error $\times$ IIT over the rows, and IIT is 0 in 90% of them. The gradient for b sums error $\times$ 1 over every row. So m's gradient is small and b's is large.

Think of the bias as the weight of a second input that is always 1. For the squared error L above, the **chain rule** (G-371; the slope of a function inside a function) gives the formulas below. Here n is the number of students, x_i the IIT value of student i, y_i the true package and $\hat{y}_i$ the predicted one.

$$\frac{\partial L}{\partial m} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\, x_i$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i) \times 1$$

A small case first: $n = 4$ students, $x = (0, 0, 0, 1)$, and every error $y_i - \hat{y}_i = 2$. Then:

$$\frac{\partial L}{\partial m} = -\frac{2}{4}\,(0 + 0 + 0 + 2 \times 1)$$

$$= -\frac{2}{4} \times 2 = -1$$

$$\frac{\partial L}{\partial b} = -\frac{2}{4}\,(2 + 2 + 2 + 2)$$

$$= -\frac{2}{4} \times 8 = -4$$

The same errors give b four times the gradient of m, because three students have $x_i = 0$. For every student with $x_i = 0$, the term in $\partial L/\partial m$ is 0. With 90 of 100 students at 0, only 10 terms remain, so the sum is small and every update of m is small. In $\partial L/\partial b$ every term counts, so the sum, and every update of b, is large. At the start $(-4, -4)$ the gradients are $\partial L/\partial m = -3.39$ and $\partial L/\partial b = -15.92$ (Notebook).

The move in the (m, b) plane is the sum of the two updates. A large update in b and a small one in m point the path almost straight along b, which is the "L" of Figure 1.

The path turns once b is close to its best value. After 10 steps of gradient descent the gradients are $\partial L/\partial b = 0.10$ and $\partial L/\partial m = -0.94$: almost nothing is left to gain along b, so the small gradient of m now sets the direction, and the path crawls along m (computed with `images/shared.py`).

To head straight for the minimum, the two updates must be of comparable size.

5. The idea: shrink the learning rate where the gradients are large

> **Key point:** The gradient cannot be changed, but the **learning rate** (G-1068) can. Give each parameter its own learning rate: small where its gradients have been large, large where they have been small.

The update is learning rate times gradient. The gradient is what it is, so **AdaGrad** (G-168) changes the other factor: a **separate learning rate for every parameter**.

- A parameter whose gradients are large, like b, gets a small learning rate, so its update stays moderate.
- A parameter whose gradients are small, like m, keeps a large learning rate, so its update is not tiny.

The two updates then become comparable, and the path heads for the minimum. AdaGrad scales each parameter's learning rate in inverse proportion to the square root of the sum of all its past squared gradients (Goodfellow et al. 2016, §8.5.1).

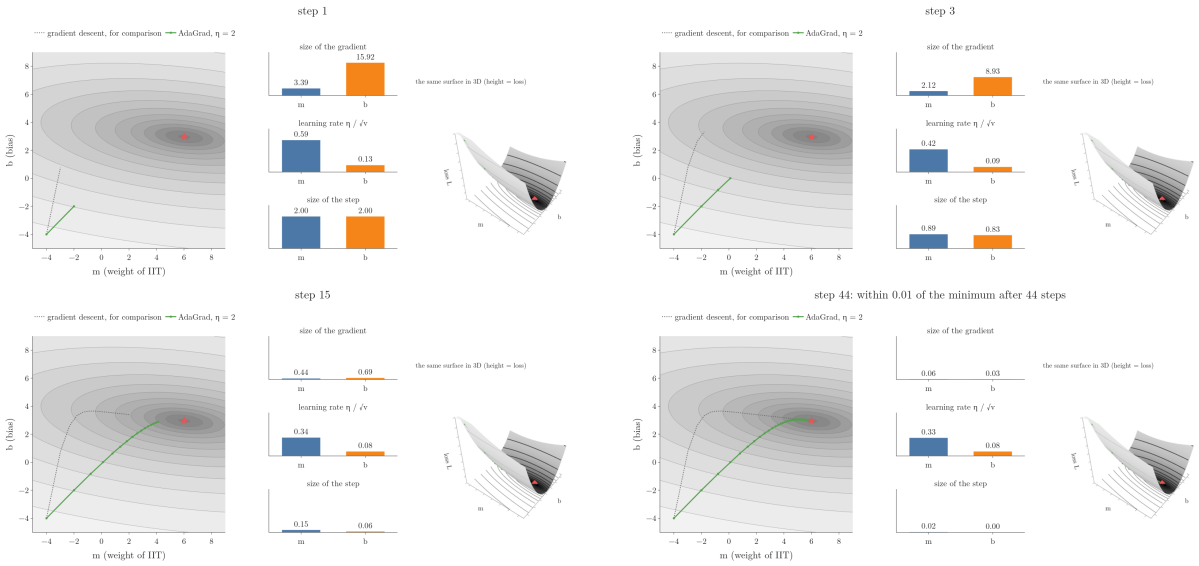


Figure 4: AdaGrad, $\eta = 2$, on the students data, step by step. Left: its path (green) beside gradient descent's (grey). Right, for m (blue) and b (orange): the size of the gradient, the learning rate $\eta/\sqrt{v}$ and the size of the update

Figure 4 shows the idea at work. Watch the right-hand bars: b's gradient is the larger one, so its sum v grows faster and its learning rate drops to about a quarter of m's. In the first steps the two updates are about the same size (2.00 and 2.00 at step 1, 0.89 and 0.83 at step 3), although b's gradient is 4 to 5 times m's, so the path heads straight for the minimum.

6. The update rule

> **Key point:** For every parameter separately, add the square of each new gradient to a running sum, and divide the **learning rate** (G-1068) by the square root of that sum before the usual gradient step.

1. **In words:** for each parameter, keep a running sum v of its squared gradients. Divide the learning rate by the square root of that sum before making the usual gradient step.
2. **Formula:**

$$v_t = v_{t-1} + (\nabla L(w_t))^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla L(w_t)$$

   with $v_0 = 0$, computed separately for every parameter. Here w_t is one parameter (such as m) after t steps, $\nabla L(w_t)$ its gradient, η the learning rate and ϵ a tiny number (below).
3. **Example:** the students data from $(m, b) = (-4, -4)$ with $\eta = 2$. The first gradients are -3.39 for m and -15.92 for b: For m:

$$v_1 = 3.39^2 = 11.5$$

$$\frac{\eta}{\sqrt{v_1}} = \frac{2}{3.39} = 0.59$$

$$\text{move} = 0.59 \times 3.39 = 2$$

   For b:

$$v_1 = 15.92^2 = 253.5$$

$$\frac{\eta}{\sqrt{v_1}} = \frac{2}{15.92} = 0.126$$

$$\text{move} = 0.126 \times 15.92 = 2$$

   On the first step, every parameter moves by exactly η, whatever the size of its gradient. Afterwards the sums differ, but m keeps a learning rate about 4 times larger than b's: 0.333 against 0.081 after 500 steps (Notebook).

The parts of the formula:

- ϵ is a tiny number, such as 10^{-7}, that only prevents division by zero when $v_t = 0$.
- v_t is the sum of all the past squared gradients of this parameter.
- **Why square?** Gradients can be positive or negative; we want their size, not their direction, so every gradient adds a positive amount.
- **Why the square root?** It brings the sum of squares back to the size of a gradient: on the first step $\sqrt{v_1}$ is exactly the size of the gradient, which is why every parameter moved by exactly η in the example above. Without the square root, **AdaGrad** (G-168) performs much worse (Ruder 2016, §4.3).

AdaGrad changes only one thing in the gradient descent update: it divides the learning rate by $\sqrt{v_t}$. A parameter with large past gradients is divided by a large number and gets a small

learning rate; one with small past gradients is divided by a small number and keeps most of it.

With this rule, AdaGrad reaches the minimum of Figure 1 in 44 steps, against 61 for gradient descent and 64 for momentum, along a path that heads for the minimum instead of making an "L" (Notebook). The learning rates differ ($\eta = 2$, 0.3 and 0.1) because each method has its own working range; the shape of the path is the point.

7. Sparse features in real data: movie reviews

> **Key point:** In text data most words are rare. On movie reviews, AdaGrad's **learning rate** (G-1068) for the rarest words ended 11 times larger than for the most common ones, and their weights 11 times larger than with SGD.

The **IMDB dataset** (G-923) of movie reviews (Maas et al. 2011) is labelled positive or negative. The setup:

- **features:** each review becomes 5,000 features; feature j is 1 if the review contains the j-th most common word, else 0;
- **data:** 10,000 reviews for training and 10,000 for validation; 97.5% of all feature values are 0, and 63% of the words appear in fewer than 1% of the reviews (Notebook);
- **model: logistic regression** (G-1120; a yes/no classifier), one **sigmoid** (G-1798; squashes the node's sum to a probability between 0 and 1) node;
- **training:** 10 **epochs** (G-696) with **batch size** (G-267) 64 and learning rate 0.1, by **SGD** (G-1892) and by **AdaGrad** (G-168), 3 seeds each; AdaGrad's sums start at 0, as in the formula.

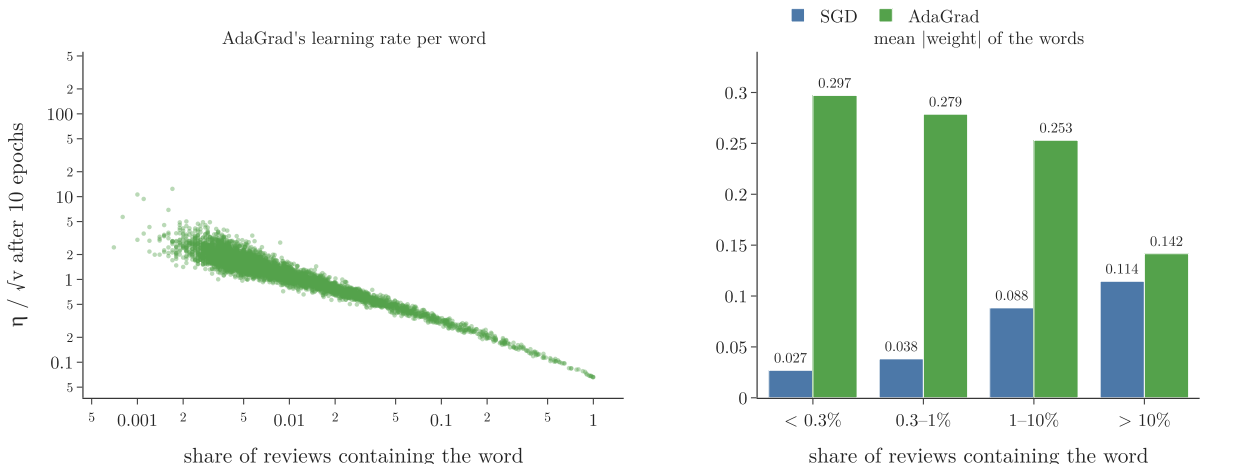


Figure 5: IMDB, 5,000 words. Left: AdaGrad's learning rate for each word after 10 epochs against the share of reviews containing the word; the rarer the word, the larger its learning rate. Right: mean size of the learned weights per frequency band; with SGD the rare words barely move

In Figure 5 (left) each dot is one word. The horizontal axis is how many reviews contain the word, the vertical axis is the learning rate AdaGrad ended with for that word; both axes are on a log scale (each gridline 10 times the previous one), so a straight line of dots means one is a fixed power of the other. The right panel groups the words by frequency and shows bars of the mean weight size.

Figure 5 and the Notebook give:

Words that appear in	Words	AdaGrad's median learning rate	Mean |weight|, SGD	Mean |weight|, AdaGrad
under 0.3% of reviews	376	2.39	0.027	0.297
0.3–1%	2,765	1.56	0.038	0.279
1–10%	1,635	0.76	0.088	0.253
over 10%	224	0.21	0.114	0.142

A rare word appears in few mini-batches, so its gradient is usually 0 and its sum v stays small; its learning rate stays large, 11 times larger for the rarest words than for the most common ones. With SGD, the same rare words get almost no updates: their weights stay near their small starting values, while AdaGrad gives them weights ten times larger.

AdaGrad also fits the training data much faster: training loss 0.099 after 10 epochs, against 0.258 for SGD. Validation accuracy is about the same, 0.864 against 0.867: on this small training set, the faster fit does not turn into better predictions on new reviews.

8. The weakness: the learning rate only goes down

> **Key point:** v_t is a sum of squares, so it never shrinks. Every step makes the **learning rates** (G-1068) smaller. Large early gradients cut them for good, and on long runs the updates can become too small to reach the minimum.

The sum v_t can only grow. So each parameter's learning rate $\eta/\sqrt{v_t}$ can only fall, and it never recovers. Large gradients early in training, when the weights are far from good values, fill v_t quickly and cut the learning rates for the rest of training.

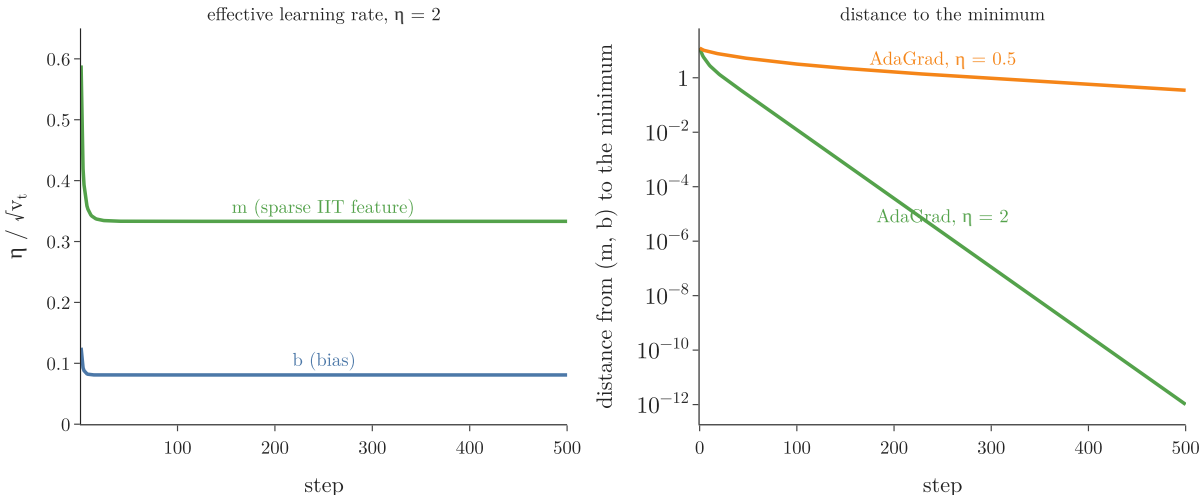


Figure 6: AdaGrad on the students data. Left: the effective learning rates of m and b with $\eta = 2$ fall in the first steps and never rise again. Right: distance to the minimum; with $\eta = 2$ AdaGrad converges, with $\eta = 0.5$ it is still 0.35 away after 500 steps

In Figure 6 the left panel shows how each parameter's learning rate changes with the step, and the right panel the distance from the minimum on a log scale (here each gridline is 100 times the one below it); a curve that goes lower is closer to the minimum.

Figure 6 shows the effect. With $\eta = 0.5$, the first large gradients cut m's learning rate from 0.147 to 0.055 within 10 steps and to 0.029 by step 500; b's falls from 0.031 to 0.007. The steps become so small that after 500 steps **AdaGrad** (G-168) is still 0.35 away from the minimum (Notebook).

For training deep networks, the accumulation of squared gradients from the beginning of training can cause a premature and excessive decrease in the **effective learning rate** (G-662; the learning rate after AdaGrad's division); AdaGrad performs well for some deep learning models but not all (Goodfellow et al. 2016, §8.5.1). Once the learning rate has become infinitesimally small, the algorithm can no longer learn anything new (Ruder 2016, §4.3). This is why AdaGrad is rarely used for complex neural networks. Its idea survives in **RMSProp** (G-1697) and **Adam** (G-169), which fix the shrinking.

> **Extra:** On a convex problem (a **convex function**, G-476) such as linear regression, AdaGrad does come with convergence guarantees (Duchi et al. 2011; Goodfellow et al. 2016, §8.5.1), and with a large enough η it converges quickly, as the $\eta = 2$ run shows. The trouble is that the right η depends on how large the early gradients happen to be.

9. AdaGrad in Keras

Key point: `keras.optimizers.Adagrad(learning_rate=...)`. Keras starts every sum at 0.1 instead of 0.

Python: AdaGrad (G-168) in Keras.

```
import keras
opt = keras.optimizers.Adagrad(learning_rate=0.01)
model.compile(optimizer=opt, loss="binary_crossentropy",
              metrics=["accuracy"])
```

Keras' defaults are `learning_rate=0.001`, `initial_accumulator_value=0.1` (the starting v_0) and `epsilon=1e-7`, which it adds inside the square root (Keras `Adagrad` documentation). Starting at 0.1 keeps the first steps from being exactly η when a gradient is tiny.

10. Summary

	Gradient descent	AdaGrad
Learning rate	one η for every parameter	$\eta/(\sqrt{v_t}+\epsilon)$, its own for each parameter
v_t	none	sum of all past squared gradients
Elongated bowl (sparse feature)	"L"-shaped path	heads for the minimum
Sparse features	rare features learn slowly	rare features get larger steps
Long runs	fixed step size	steps keep shrinking, may stall

- AdaGrad divides the **learning rate** (G-1068) by the square root of the sum of past squared gradients, per parameter, because the gradient cannot be changed but the learning rate can.
- Large past gradients mean a small learning rate; small ones keep it large. Updates in all directions become comparable, so the path heads for the minimum instead of making an "L".
- It suits sparse data, where rare features need larger updates: a rare feature's gradient is usually 0, so its sum stays small and its learning rate stays large.
- Its learning rates only fall, because v_t is a sum of squares and never shrinks; large early gradients cut them for good, which makes it unsuited to most deep networks.
- RMSProp and Adam keep the idea and fix the shrinking, so **AdaGrad** (G-168) itself is rarely used for complex neural networks.

11. Sources

Built from

- CampusX, "AdaGrad Explained in Detail with Animations | Optimizers in Deep Learning Part 4", YouTube, https://www.youtube.com/watch?v=nqL9xYmhEpg

Other references

- Duchi, J., Hazan, E. and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research* 12, 2121–2159.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §8.5 and §8.5.1 AdaGrad (algorithm 8.4).
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv:1609.04747. §4.3 Adagrad.
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y. and Potts, C. (2011). Learning word vectors for sentiment analysis. ACL 2011 (the IMDB review dataset).
- Keras API documentation: `Adagrad` optimizer, keras.io/api/optimizers/adagrad.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Sparse feature (G-1841)	A feature whose values are mostly zero, such as "studied at an IIT".
AdaGrad (G-168)	An optimizer that gives each parameter its own learning rate, $\eta/(\sqrt{v_t}+\epsilon)$, where v_t sums its past squared gradients.
Elongated bowl (G-673)	A loss surface stretched far more in one direction than another, with long elliptical contours, as a sparse feature produces. Gradient descent on it wastes steps moving along the steep direction and crawls along the flat one.
Effective learning rate (G-662)	The learning rate a parameter actually gets after AdaGrad's division: $\eta/(\sqrt{v_t}+\epsilon)$.
Adaptive learning rate (G-171)	A learning rate that changes during training according to the gradients seen so far.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Batch gradient descent (G-264)	Gradient descent that uses all training rows for every update.
Momentum (optimizer) (G-1258)	Gradient descent that keeps part of its past movement: each step follows an average of past gradients in which older ones count less and less (a velocity, an exponentially decaying average).
Nesterov accelerated gradient (NAG) (G-1315)	A version of momentum that first takes the momentum jump and then measures the gradient where it lands (the look-ahead point); seeing the slope ahead lets it brake before the minimum and swing less.
Overshooting (G-1430)	Moving past the minimum because of the built-up velocity, then swinging back.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.

Term	Meaning
Sparse data (G-1840)	Two senses: a table that is mostly zeros (as after one-hot encoding); or, in many dimensions, a space where most regions hold no points.
Observation (G-1374)	One record of a dataset: one row of the data table.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
IMDB dataset (G-923)	50,000 film reviews labelled positive or negative, a standard sentiment-analysis dataset.
Logistic regression (G-1120)	A classification algorithm that passes a weighted sum of the inputs through the sigmoid to give the probability of a class; the line (or plane) where that probability is 0.5 is the boundary it learns between the classes.
Sigmoid function (G-1798)	An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1 + e^{-z})$; it turns a score into a probability, as in logistic regression.
Epoch (G-696)	One full update of the parameters using the whole training set.
Batch size (G-267)	The number of rows in each batch; a hyperparameter.
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.
RMSProp (G-1697)	An optimizer that divides each parameter's step by the root of a running average (EWMA) of its recent squared gradients, $v_t = \beta v_{t-1} + (1 - \beta) g_t^2$; old gradients fade, so unlike AdaGrad the learning rate does not shrink to nothing.
Adam (G-169)	An optimizer, the rule that updates a network's weights to reduce the loss: a variant of gradient descent that keeps running averages of past gradients and of their squares, giving each weight its own step size; fairly robust to its settings, so a common default.

Term	Meaning
Convex function (G-476)	A function where a straight line between any two points of its curve never goes below the curve; it has a single minimum.
ϵ (epsilon) (G-9)	A tiny number added to the variance to avoid dividing by 0; 0.001 in Keras.